K.R. MANGALAM UNIVERSITY

Database Management Lab File (ENCS205)

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

Gaurav (2401010009)

B. Tech CSE (Core) – 4th Semester

Section: A

Submitted To:

MS. Mansi Kajal

School of Engineering & Technology
K. R. MANGALAM UNIVERSITY

Sohna, Haryana 122103, India Jan-2026

EXPERIMENT:3

3.Database Schema for a customer-sale scenario

Customer (Cust id: integer, cust_name: string)

Item (item_id: integer, item_name: string, price: integer)

Sale (bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

2. Insert around 10 records in each of the tables.

3. List all the bills for the current date with the customer names and item numbers.

4. List the total Bill details with the quantity sold, price of the item and the final amount.

5. List the details of the customer who have bought a product which has a price>200.

6. Give a count of how many products have been bought by each customer

7. Give a list of products bought by a customer having cust_id as 5.

8. List the item details which are sold as of today.

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

10. Create a view which lists the daily sales date wise for the last one week

```
CREATE DATABASE IF NOT EXISTS sales_db;
USE sales_db;

DROP VIEW IF EXISTS Weekly_Sales;
DROP VIEW IF EXISTS Bill_Details;

DROP TABLE IF EXISTS Sale;
DROP TABLE IF EXISTS Item;
DROP TABLE IF EXISTS Customer;

CREATE TABLE Customer (
   cust_id INT PRIMARY KEY,
   cust_name VARCHAR(50) NOT NULL
   );

CREATE TABLE Item (
   item_id INT PRIMARY KEY,
   item_name VARCHAR(50) NOT NULL,
   price INT NOT NULL CHECK (price > 0)
);

CREATE TABLE Sale (
   bill_no INT,
   bill_date DATE NOT NULL,
   cust_id INT,
   item_id INT,
   qty_sold INT NOT NULL CHECK (qty_sold > 0),
   PRIMARY KEY (bill_no, item_id),
   FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
```

```
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
INSERT INTO Customer VALUES
(1,'Gaurav'),
(2,'Rahul'),
(3,'Sneha'),
(4,'Priya'),
(5,'Karan'),
(6,'Neha'),
(7,'Vikas'),
(8,'Rohit'),
(9,'Pooja'),
(10,'Anjali');

INSERT INTO Item VALUES
(101,'Pen',50),
(102,'Notebook',120),
(103,'Bag',500),
(104,'Bottle',250),
(105,'Shoes',1500),
(106,'Pencil',20),
(107,'Scale',30),
(108,'Watch',2000),
(109,'T-shirt',800),
(110,'Headphones',1800);

INSERT INTO Sale VALUES
(1000,CURRENT_DATE,1,101,2),
(1001,CURRENT_DATE,1,103,1),
```

```
(1002,CURRENT_DATE,2,105,1),
(1003,CURRENT_DATE,3,104,3),
(1004,DATE_SUB(CURRENT_DATE, INTERVAL 1 DAY),5,108,1),
(1005,DATE_SUB(CURRENT_DATE, INTERVAL 2 DAY),5,102,4),
(1006,CURRENT_DATE,6,109,2),
(1007,CURRENT_DATE,7,110,1),
(1008,DATE_SUB(CURRENT_DATE, INTERVAL 3 DAY),8,103,1),
(1009,CURRENT_DATE,9,104,2);

SELECT s.bill_no, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURRENT_DATE;

SELECT s.bill_no,
    s.item_id,
    s.qty_sold,
    i.price,
    (s.qty_sold * i.price) AS Final_Amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;

SELECT DISTINCT c.*
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;

SELECT c.cust_name,
```

```
    SUM(s.qty_sold) AS Total_Products
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_name;

SELECT i.item_name
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;

SELECT DISTINCT i.*
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURRENT_DATE;

CREATE VIEW Bill_Details AS
SELECT s.bill_no,
    s.bill_date,
    s.cust_id,
    s.item_id,
    i.price,
    s.qty_sold,
    (s.qty_sold * i.price) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;

CREATE VIEW Weekly_Sales AS
SELECT s.bill_date,
    SUM(s.qty_sold * i.price) AS Total_Sales
```

```
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date >= DATE_SUB(CURRENT_DATE, INTERVAL 7 DAY)
GROUP BY s.bill_date;

SELECT * FROM Bill_Details;
SELECT * FROM Weekly_Sales;
```

OUTPUT:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	cust_name	item_id
1000	Gaurav	101
1001	Gaurav	103
1002	Rahul	105
1003	Sneha	104
1006	Neha	109
1007	Vikas	110
1009	Pooja	104

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	item_id	qty_sold	price	Final_Amount
1000	101	2	50	100
1001	103	1	500	500
1002	105	1	1500	1500
1003	104	3	250	750
1004	108	1	2000	2000
1005	102	4	120	480
1006	109	2	800	1600
1007	110	1	1800	1800
1008	103	1	500	500
1009	104	2	250	500

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_id	cust_name
1	Gaurav
8	Rohit
3	Sneha
9	Pooja
2	Rahul
5	Karan
6	Neha
7	Vikas

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_name	Total_Products
Gaurav	3
Rahul	1
Sneha	3
Karan	5
Neha	2
Vikas	1
Rohit	1
Pooja	2

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

item_name
Watch
Notebook

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

item_id	item_name	price
101	Pen	50
103	Bag	500
105	Shoes	1500
104	Bottle	250
109	T-shirt	800
110	Headphones	1800

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1000	2026-03-01	1	101	50	2	100
1001	2026-03-01	1	103	500	1	500
1002	2026-03-01	2	105	1500	1	1500
1003	2026-03-01	3	104	250	3	750
1004	2026-02-28	5	108	2000	1	2000
1005	2026-02-27	5	102	120	4	480
1006	2026-03-01	6	109	800	2	1600
1007	2026-03-01	7	110	1800	1	1800
1008	2026-02-26	8	103	500	1	500
1009	2026-03-01	9	104	250	2	500

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_date	Total_Sales
2026-03-01	6750
2026-02-28	2000
2026-02-27	480
2026-02-26	500